

1. Reverse each word in a given string while keeping the original word order

```
String input = "Java Stream API";

String result = Arrays.stream(input.split(" "))
    .map(word -> new StringBuilder(word).reverse().toString())
    .collect(Collectors.joining(" "));

System.out.println(result);
```

⚙️ **Output:** avaJ maertS IPA

2. Count the length of each word in a String ArrayList using Java 8

```
List<String> words = Arrays.asList("A", "CC", "SASAS");

Map<String, Integer> wordLen = words.stream()
    .collect(Collectors.toMap(Function.identity(), String::length));

System.out.println(wordLen);
```

⚙️ **Output:** {A=1, CC=2, SASAS=5}

3. Find the longest word in a given sentence using Java 8 Streams

```
String input = "Java Stream API is powerful";

String longest = Arrays.stream(input.split(" "))
    .max(Comparator.comparingInt(String::length))
    .orElse("");

System.out.println(longest);
```

⚙️ Output: **powerful**

4. Rearrange the characters of a given string in alphabetical order using Streams

```
String input = "banana";

String sorted = input.chars()
    .sorted()
    .mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining());

System.out.println(sorted);
```

⚙️ Output: **aaabnn**

5. Arrange letters of each word in alphabetical order while keeping the word order

Java 8 Stream Solution:

```
String input = "java stream api";

String result = Arrays.stream(input.split(" "))
    .map(word -> word.chars()
        .sorted()
        .mapToObj(c -> String.valueOf((char) c))
        .collect(Collectors.joining()))
    .collect(Collectors.joining(" "));

System.out.println(result);
```

 **Output:** aajv aemrst aip

6. Count the frequency of each element/word from a String ArrayList using Java 8

```
List<String> words = Arrays.asList("apple", "banana", "apple",
    "cherry", "banana");

Map<String, Long> freq = words.stream()
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()));

System.out.println(freq);
```

 **Output:** {banana=2, cherry=1, apple=2}

7. Count the frequency of each character in a given string using Java 8

```
String input = "testjavadeveloper";

Map<Character, Long> freq = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
        LinkedHashMap::new, Collectors.counting()));
freq.forEach((ch, count) -> System.out.println(ch + " = " + count));
```

⚙️ **Output:**

```
t = 2,e = 3,s = 1,j = 1,a = 2,v = 2,
d = 1,l = 1,o = 1,p = 1,r = 1
```

8. Find and print all duplicate characters in a string

```
String input = "programming";

Set<Character> duplicates = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
        Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
System.out.println(duplicates);
```

⚙️ **Output:** [r, g, m]

9. Find the first repeated character in a given string using Stream functions

```
String input = "swiss";
```

```
Optional<Character> firstRepeat = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
    LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .findFirst();
firstRepeat.ifPresent(System.out::println);
```

⚙️ Output: **s**

10. Find the first non-repeated character in a given string using Stream functions

```
String input = "swiss";
```

```
Optional<Character> firstUnique = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
    LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst();
firstUnique.ifPresent(System.out::println);
```

⚙️ Output: **w**

11. Find the last non-repeated (unique) character in a given string

```
String input = "swiss";
Map<Character, Long> freq = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
        LinkedHashMap::new, Collectors.counting()));

Character lastNonOccur = freq.entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .reduce((first, second) -> second)
    .orElse(null);
System.out.println(lastNonOccur);
```

⚙️ **Output:**i

12. Find the most frequently occurring character in a given string using Java 8 Streams

✖️ **Problem:**

Find the character that occurs most frequently in the string.

♦ **Example:**

Input: "success" -> **Output:** s

💻 **Java 8 Stream Solution:**

```
String input = "success";

Character maxOccur = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
        Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null);
System.out.println(maxOccur);
```

⚙️ **Output:**s

13. Find the character with the second-highest frequency using Java 8 Streams

```
String input = "success";

Character secondMax = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<Character, Long>comparingByValue().reversed())
    .skip(1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);

System.out.println(secondMax);
```

⚙️ Output: **c**